

Q3

October 19, 2024

0.0.1 Question 3: A ML model to distinguish spam and non-spam E-mails

First Step: Preprocessing the data

At the first step we import the nltk, pandas and numpy modules to use their practical functions for preprocessing and analysing the data.

```
[2]: import nltk
import numpy as np
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import pandas as pd
import string
```

First we read the emails texts and labels from the csv file.

```
[3]: emails_data = pd.read_csv('emails.csv')
```

Then we lowercase the letters in the text, omit the punctuation marks and numbers from them and tokenize the words.

```
[4]: emails_data['text'] = emails_data['text'].str.lower()
emails_data['text'] = emails_data['text'].str.translate(str.maketrans('', '',
    ↪string.punctuation + '0123456789'))
emails_data['text'] = emails_data['text'].apply(word_tokenize)
emails_data['text'] = emails_data['text'].apply(lambda words: ' '.join(words))
    ↪#Vectorizer gets string as input.
```

Second Step: Splitting the data

In this step we only need to split the data and produce train and test sets. For this purpose we need scikit learn module and specifically its famous function “train_test_split()”

```
[5]: from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(emails_data['text'],
    ↪emails_data['spam'], test_size=0.2, random_state=235)
```

Third Step: Making BoW from x_train

Now we make BoW matrix. It is a $2 \times \text{num_of_distinct_words}$ matrix which implies the number of occurrences for each word in the whole spam or non-spam texts. BoW[0] is for the non-spam texts and BoW[1] is for the spam texts.

We use the “CountVectorizer()” function from scikit learn to vectorize the email texts and have a practical dictionary of all of distinct words in the training set.

```
[6]: from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()
features_list = vectorizer.fit_transform(x_train).toarray().tolist()
num_of_distinct_words = len(features_list[0])
BoW_train = np.zeros((2, num_of_distinct_words), dtype=int)
train_labels = y_train.to_numpy()
```

Now we fill the BoW by summing over the features list.

```
[7]: for i in range(len(features_list)):
      if train_labels[i]:
          BoW_train[1] += features_list[i]
      else:
          BoW_train[0] += features_list[i]
```

Now we calculate $P(x_i|y)$ for each word if spam or non-spam.

```
[8]: num_of_whole_words_spam = np.sum(BoW_train[1])
num_of_whole_words_non_spam = np.sum(BoW_train[0])
words_p_of_happening_if_spam = []
words_p_of_happening_if_non_spam = []

for i in range(num_of_distinct_words):
    words_p_of_happening_if_spam.append(BoW_train[1][i] /
    ↪ num_of_whole_words_spam)
    words_p_of_happening_if_non_spam.append(BoW_train[0][i] /
    ↪ num_of_whole_words_non_spam)
```

Now we calculate $P(y)$ for each class(spam and non-spam).

```
[9]: num_of_spams = 0
for i in range(train_labels.size):
    if train_labels[i] == 1:
        num_of_spams += 1

general_p_of_being_spam = num_of_spams / len(train_labels)
general_p_of_being_non_spam = 1 - general_p_of_being_spam
```

Forth Step: Making BoW from x_train

Now we have to predict whether an E-mail is spam or not using the naive Bayes classifier. To do this we implement the Bayes rule in a function named is_spam and use this function over all the x_test.

Then to evaluate the model we write an evaluation function and evaluate it.

```
[20]: def is_spam(features : list, general_spam_p, general_non_spam_p, num_of_words,
        words_p_of_happening_if_spam, words_p_of_happening_if_non_spam):
        spam_probability, non_spam_probability = 1, 1

        for i in range(num_of_words):
            spam_probability *= words_p_of_happening_if_spam[i] ** features[i] if
            features[i] != 0 else 1
            non_spam_probability *= words_p_of_happening_if_non_spam[i] **
            features[i] if features[i] != 0 else 1

        spam_probability *= general_spam_p
        non_spam_probability *= general_non_spam_p

        return int(spam_probability > non_spam_probability)

def evaluate_model(predictions, y_test):
    num_of_correct_predictions = 0
    num_of_predictions = len(predictions)

    for i in range(num_of_predictions):

        if predictions[i] == y_test[i]:
            num_of_correct_predictions += 1

    return num_of_correct_predictions / num_of_predictions
```

Now we predict using a loop and and evaluate the model.

```
[28]: test_data_features_list = vectorizer.transform(x_test).toarray().tolist()
num_of_predictions = len(test_data_features_list)
predictions = [0] * num_of_predictions

for i in range(num_of_predictions):
    predictions[i] = is_spam(test_data_features_list[i],
        general_p_of_being_spam, general_p_of_being_non_spam,
        num_of_distinct_words,
        words_p_of_happening_if_spam, words_p_of_happening_if_non_spam)

print(evaluate_model(predictions, y_test.to_numpy()))
```

0.8525305410122164

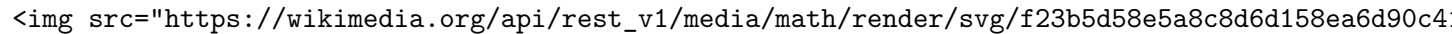
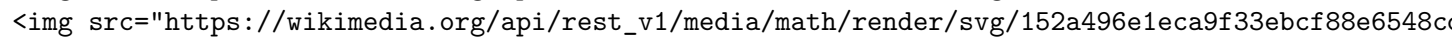
Q1:

According to Laplace smoothing method (additive smoothing), we should prevent the probabilities from becoming equal to 0; because it'll make the whole probability 0.

In the other hand, if we consider its value equal to 1, we are ignoring that the testing text has a

word which doesn't exist in the whole BoW. This way the unknown words won't involve in deciding and will not have any effect on the result. So we use smoothing to be sure that even the unknown words play a role in classifying.

To do this, we add a specific value to each probability. There are two situations: 1. The probability of a chosen word in the spam's BoW is 0 or 2. One of the words in the testing E-mail does not exist in the BoW. We consider the smoothing for both situations.

```
[30]: alpha = 1
d = num_of_distinct_words
words_p_of_happening_if_spam_smoothed = []
words_p_of_happening_if_non_spam_smoothed = []

for i in range(num_of_distinct_words):
    p_spam_smoothed = (BoW_train[1][i] + alpha) / (num_of_whole_words_spam + d
    ↪ * alpha)
    words_p_of_happening_if_spam_smoothed.append(p_spam_smoothed)

    p_non_spam_smoothed = (BoW_train[0][i] + alpha) /
    ↪ (num_of_whole_words_non_spam + d * alpha)
    words_p_of_happening_if_non_spam_smoothed.append(p_non_spam_smoothed)

def is_spam_smoothed(features : list, general_spam_p, general_non_spam_p,
    ↪ num_of_words,
        words_p_of_happening_if_spam, words_p_of_happening_if_non_spam,
        unknown_word_p_spam, unknown_word_p_non_spam, num_of_unknown_words):

    spam_probability, non_spam_probability = 1, 1

    for i in range(num_of_words):
        spam_probability *= words_p_of_happening_if_spam[i] ** features[i] if
        ↪ features[i] != 0 else 1
        non_spam_probability *= words_p_of_happening_if_non_spam[i] **
        ↪ features[i] if features[i] != 0 else 1

    spam_probability *= general_spam_p
    spam_probability *= unknown_word_p_spam ** num_of_unknown_words
    non_spam_probability *= general_non_spam_p
    non_spam_probability *= unknown_word_p_non_spam ** num_of_unknown_words

    return int(spam_probability > non_spam_probability)
```

Now we evaluate:

```
[34]: words_count_test = [0] * num_of_predictions
for i, x in enumerate(x_test):
    words_count_test[i] = len(x.split(' '))

unknown_word_p_spam = alpha / (num_of_whole_words_spam + alpha * d)
unknown_word_p_non_spam = alpha / (num_of_whole_words_non_spam + alpha * d)
for i in range(num_of_predictions):
    predictions[i] = is_spam_smoothed(test_data_features_list[i],
    ↪general_p_of_being_spam, general_p_of_being_non_spam,
    ↪num_of_distinct_words,
    ↪words_p_of_happening_if_spam_smoothed,
    ↪words_p_of_happening_if_non_spam_smoothed,
    ↪unknown_word_p_spam, unknown_word_p_non_spam,
    ↪words_count_test[i] - np.sum(test_data_features_list[i]))

print(evaluate_model(predictions, y_test.to_numpy()))
```

0.8839441535776614

Q2:

If the number of words in the text is so high, the value of probabilities will underflow; because a lot of numbers less than 1 are being multiplied and get smaller and the computer has limitations to calculate fractional numbers.

To solve this, we can use logarithm. Because logarithm is a strictly ascending function in the inequality we can apply it on both sides and the result won't change. The advantage is that log converts multiplying to summation. So the result won't get smaller.

```
[32]: def is_spam_smoothed_log(features : list, general_spam_p, general_non_spam_p,
    ↪num_of_words,
    ↪words_p_of_happening_if_spam, words_p_of_happening_if_non_spam,
    ↪unknown_word_p_spam, unknown_word_p_non_spam, num_of_unknown_words):
    spam_probability, non_spam_probability = 0, 0

    for i in range(num_of_words):
        spam_probability += features[i] * np.
    ↪log(words_p_of_happening_if_spam[i] if features[i] != 0 else 1)
        non_spam_probability += features[i] * np.
    ↪log(words_p_of_happening_if_non_spam[i] if features[i] != 0 else 1)

    spam_probability += np.log(general_spam_p)
    spam_probability += num_of_unknown_words * np.log(unknown_word_p_spam)
    non_spam_probability += np.log(general_non_spam_p)
    non_spam_probability += num_of_unknown_words * np.
    ↪log(unknown_word_p_non_spam)

    return int(spam_probability > non_spam_probability)
```

Now we evaluate it:

```
[33]: for i in range(num_of_predictions):
        predictions[i] = is_spam_smoothed_log(test_data_features_list[i],
        ↪general_p_of_being_spam, general_p_of_being_non_spam,
        ↪num_of_distinct_words,
        ↪words_p_of_happening_if_spam_smoothed,
        ↪words_p_of_happening_if_non_spam_smoothed,
        ↪unknown_word_p_spam, unknown_word_p_non_spam,
        ↪words_count_test[i] - np.sum(test_data_features_list[i]))

    print(evaluate_model(predictions, y_test.to_numpy()))
```

0.981675392670157

Q3:

Using the “stopwords” table in nltk, we can delete the stop words from our data. I also use stemmer to delete the extra words which have the same roots to the formers. To do this, I have to run most of the codes again!

```
[39]: nltk.download('stopwords')

emails_data['text'] = emails_data['text'].apply(word_tokenize)
stop_words = stopwords.words('english')
emails_data['text'] = emails_data['text'].apply(lambda words: [word for word in
    ↪words if word not in stop_words])
stemmer = PorterStemmer()
emails_data['text'] = emails_data['text'].apply(lambda words: [stemmer.
    ↪stem(word) for word in words])
emails_data['text'] = emails_data['text'].apply(lambda words: ' '.join(words))

x_train, x_test, y_train, y_test = train_test_split(emails_data['text'],
    ↪emails_data['spam'], test_size=0.2, random_state=235)
vectorizer = CountVectorizer()
features_list = vectorizer.fit_transform(x_train).toarray().tolist()
num_of_distinct_words = len(features_list[0])
BoW_train = np.zeros((2, num_of_distinct_words), dtype=int)
train_labels = y_train.to_numpy()

for i in range(len(features_list)):
    if train_labels[i]:
        BoW_train[1] += features_list[i]
    else:
        BoW_train[0] += features_list[i]

num_of_whole_words_spam = np.sum(BoW_train[1])
num_of_whole_words_non_spam = np.sum(BoW_train[0])
```

```

words_p_of_happening_if_spam_smoothed = []
words_p_of_happening_if_non_spam_smoothed = []

alpha = 1
d = num_of_distinct_words

for i in range(num_of_distinct_words):
    p_spam = (BoW_train[1][i] + alpha) / (num_of_whole_words_spam + d * alpha)
    words_p_of_happening_if_spam_smoothed.append(p_spam)

    p_non_spam = (BoW_train[0][i] + alpha) / (num_of_whole_words_non_spam + d *
↪alpha)
    words_p_of_happening_if_non_spam_smoothed.append(p_non_spam)

num_of_spams = 0
for i in range(train_labels.size):
    if train_labels[i] == 1:
        num_of_spams += 1

general_p_of_being_spam = num_of_spams / len(train_labels)
general_p_of_being_non_spam = 1 - general_p_of_being_spam

test_data_features_list = vectorizer.transform(x_test).toarray().tolist()
num_of_predictions = len(test_data_features_list)
predictions = [0] * num_of_predictions

words_count_test = [0] * num_of_predictions
for i, x in enumerate(x_test):
    words_count_test[i] = len(x.split(' '))

unknown_word_p_spam = alpha / (num_of_whole_words_spam + alpha * d)
unknown_word_p_non_spam = alpha / (num_of_whole_words_non_spam + alpha * d)

```

```

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Shahab\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

Now we evaluate the final model:

As you can see, the final accuracy is better than the accuracy without deleting the stopwords.

```

[40]: for i in range(num_of_predictions):
        predictions[i] = is_spam_smoothed_log(test_data_features_list[i],
↪general_p_of_being_spam, general_p_of_being_non_spam,
                                num_of_distinct_words,
↪words_p_of_happening_if_spam_smoothed,
↪words_p_of_happening_if_non_spam_smoothed,

```

```
                unknown_word_p_spam, unknown_word_p_non_spam,
↪ words_count_test[i] - np.sum(test_data_features_list[i]))
print(evaluate_model(predictions, y_test.to_numpy()))
```

0.987783595113438